

Checkpoint

1. 代码实现了哪种空闲链表？ 哪种搜索方式？
2. 代码中实现的 malloc 是多少字节对齐的？
3. 一个字为多少字节？一个双字呢？
4. 给出header的布局图。

内存初始化模块

```
int mm_init(void)
{
    if ((heap_listp = mem_sbrk(4*WSIZE)) == (void *)-1)
        return -1;
    PUT(heap_listp, 0); /* 对齐填充 */
    PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1)); /* 头部 */
    PUT(heap_listp + (2*WSIZE), PACK(0, 1)); /* 脚部 */
    heap_listp += (2*WSIZE);

    if (extend_heap(CHUNKSIZE/WSIZE) == NULL)
        return -1;
    return 0;
}
```

1. heap_listp = mem_sbrk(4*WSIZE)这行代码的作用是什么？ mem_sbrk函数的功能是什么
2. PUT(heap_listp, 0);这行代码的作用是什么？ 这里为什么要进行对齐填充？
3. PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1));这行代码的作用是什么？ 这里的PACK(DSIZE, 1) 是什么含义？
4. heap_listp += (2*WSIZE);这行代码的作用是什么？ 为什么要将heap_listp向前移动两个字的大小？
5. 请解释mm_init函数的整体工作流程。它是如何初始化内存管理器的？

内存分配模块

```

void *mm_malloc(size_t size)
{
    size_t asize;
    size_t extendsize;
    char *bp;

    if (size == 0)
        return NULL;

    if (size <= DSIZE)
        asize = 2*DSIZE;
    else
        asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);

    if ((bp = find_fit(asize)) != NULL) {
        place(bp, asize);
        return bp;
    }

    extendsize = MAX(asize, CHUNKSIZE);
    if ((bp = extend_heap(extendsize/WSIZE)) == NULL)
        return NULL;
    place(bp, asize);
    return bp;
}

```

1. if (size <= DSIZE) asize = 2*DSIZE; else asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);
这两行代码的作用是什么？这里为什么要对size进行调整？
2. if ((bp = find_fit(asize)) != NULL) { place(bp, asize); return bp; }这行代码的作用是什么？
find_fit函数和place函数的功能是什么？ /
3. extendsize = MAX(asize, CHUNKSIZE);这行代码的作用是什么？为什么要计算extendsize？
4. 请解释mm_malloc函数的整体工作流程。它是如何分配内存的？
5. 如果在空闲列表中找不到合适的块，mm_malloc函数会如何处理？

内存释放模块

```

void mm_free(void *bp)
{
    if (bp == 0)
        return;

    size_t size = GET_SIZE(HDRP(bp));

    PUT(HDRP(bp), PACK(size, 0));
    PUT(FTRP(bp), PACK(size, 0));
    coalesce(bp);
}

```

1. `size_t size = GET_SIZE(HDRP(bp));`这行代码的作用是什么？`GET_SIZE`和`HDRP`这两个宏的功能是什么？
2. `PUT(HDRP(bp), PACK(size, 0));` `PUT(FTRP(bp), PACK(size, 0));`这两行代码的作用是什么？这里为什么要修改块的头部和尾部？
3. `coalesce(bp);`这行代码的作用是什么？`coalesce`函数的功能是什么？
4. 请解释`mm_free`函数的整体工作流程。它是如何释放内存的？
5. 如果相邻的块也是空闲的，`mm_free`函数会如何处理？

堆扩展模块

```

static void *extend_heap(size_t words)
{
    char *bp;
    size_t size;

    size = (words % 2) ? (words+1) * WSIZE : words * WSIZE;
    if ((long)(bp = mem_sbrk(size)) == -1)
        return NULL;

    PUT(HDRP(bp), PACK(size, 0));
    PUT(FTRP(bp), PACK(size, 0));
    PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1));

    return coalesce(bp);
}

```

1. `size = (words % 2) ? (words+1) * WSIZE : words * WSIZE;`这行代码的作用是什么？这里为什么要检查`words`是否为奇数？
2. `PUT(HDRP(bp), PACK(size, 0));` `PUT(FTRP(bp), PACK(size, 0));`这两行代码的作用是什么？这里为什么要修改新扩展块的头部和尾部？

3. `PUT(HDRP(NEXT_BLK(b)), PACK(0, 1));`这行代码的作用是什么？这里为什么要修改下一个块的头部？
4. `return coalesce(b);`这行代码的作用是什么？`coalesce`函数的功能是什么？
5. 请解释`extend_heap`函数的整体工作流程。它是如何扩展堆的？

块放置模块

```
static void place(void *bp, size_t asize)
{
    size_t csize = GET_SIZE(HDRP(bp));

    if ((csize - asize) >= (2*DSIZE)) {
        PUT(HDRP(bp), PACK(asize, 1));
        PUT(FTRP(bp), PACK(asize, 1));
        bp = NEXT_BLK(bp);
        PUT(HDRP(bp), PACK(csize-asize, 0));
        PUT(FTRP(bp), PACK(csize-asize, 0));
    }
    else {
        PUT(HDRP(bp), PACK(csize, 1));
        PUT(FTRP(bp), PACK(csize, 1));
    }
}
```

1. `if ((csize - asize) >= (2 * DSIZE)) { ... } else { ... }`这段代码的作用是什么？这里为什么要检查 `csize - asize`是否大于等于 `2 * DSIZE`？
2. `PUT(HDRP(bp), PACK(asize, 1)); PUT(FTRP(bp), PACK(asize, 1));`这两行代码的作用是什么？这里为什么要修改块的头部和尾部？
3. `bp = NEXT_BLK(bp); PUT(HDRP(bp), PACK(csize-asize, 0)); PUT(FTRP(bp), PACK(csize-asize, 0));`这三行代码的作用是什么？这里为什么要修改下一个块的头部和尾部？
4. 请解释`place`函数的整体工作流程。它是如何在空闲块中放置请求的数据的？

查找模块

```
static void *find_fit(size_t asize)
{
    void *bp;

    for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKp(bp)) {
        if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))) {
            return bp;
        }
    }
    return NULL;
}
```

1. for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKp(bp)) { ... }这段代码的作用是什么？这里为什么要遍历整个堆？
2. if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))) { ... }这段代码的作用是什么？这里为什么要检查块是否已分配以及asize是否小于等于块的大小？
3. 请解释find_fit函数的整体工作流程。它是如何在空闲列表中寻找一个大小合适的块的？

合并空闲块模块

```

static void *coalesce(void *bp)
{
    size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKPTR(bp)));
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKPTR(bp)));
    size_t size = GET_SIZE(HDRP(bp));

    if (prev_alloc && next_alloc) {           /* Case 1 */
        return bp;
    }

    else if (prev_alloc && !next_alloc) {      /* Case 2 */
        size += GET_SIZE(HDRP(NEXT_BLKPTR(bp)));
        PUT(HDRP(bp), PACK(size, 0));
        PUT(FTRP(bp), PACK(size, 0));
    }

    else if (!prev_alloc && next_alloc) {      /* Case 3 */
        size += GET_SIZE(HDRP(PREV_BLKPTR(bp)));
        PUT(FTRP(bp), PACK(size, 0));
        PUT(HDRP(PREV_BLKPTR(bp)), PACK(size, 0));
        bp = PREV_BLKPTR(bp);
    }

    else {                                     /* Case 4 */
        size += GET_SIZE(HDRP(PREV_BLKPTR(bp))) +
            GET_SIZE(FTRP(NEXT_BLKPTR(bp)));
        PUT(HDRP(PREV_BLKPTR(bp)), PACK(size, 0));
        PUT(FTRP(NEXT_BLKPTR(bp)), PACK(size, 0));
        bp = PREV_BLKPTR(bp);
    }

    return bp;
}

```

1. 请解释coalesce函数的整体工作流程。它是如何合并空闲块的？
2. 对于每一种case，给出它的内存示意图

优化 malloc

这段代码实现了一个基本的内存管理器，它使用了隐式空闲链表和首次适应策略。虽然这个内存管理器的实现是正确的，但是在性能上还有很多可以优化的地方。你能想到哪些可以优化的地方，用怎样的方式去优化？（可选：给出优化代码）